

Week 2 · Class 2: Practice Exercises — ANSWER KEY

Working with Real Data & Summary Statistics

2025-12-31

1 Segment A — Working with Real Data

2 Non-AI Exercises

2.1 1. Working with Missing Data

2.1.1 1.1 The Challenge of Missing Data

Why can't we use `== NA` to check for missing values in R? What does this tell us about how R thinks about missing data?

Answer: **We cannot use `== NA` because `NA` represents an unknown value, and comparing anything to an unknown value returns `NA`, not `TRUE` or `FALSE`.** This tells us that R treats missing data as truly “unknown” - it's not zero, it's not empty, it's simply unknown. When you ask “is this value equal to `NA`?”, R responds with “I don't know” (which is `NA`). This is why we need the special function `is.na()` which is specifically designed to detect missing values.

2.1.2 1.2 Code Detective: Missing Data

Explain what's wrong with this code and how to fix it:

```
data %>%  
  filter(age == NA)
```

Problem: The expression `age == NA` will always return `NA`, not `TRUE` or `FALSE`, so the filter won't work properly. It will return zero rows regardless of how many missing values exist. Use `is.na(age)` instead: `data %>% filter(is.na(age))` to keep rows with missing age values, or `data %>% filter(!is.na(age))` to remove them.

2.2 2. The Pipe Operator

2.2.1 2.1 Pipe vs No Pipe

These two pieces of code do the same thing. Which is easier to read and why?

```
# Version A
arrange(select(filter(congress, party_code == "Republican"),
  bioname, nominate_dim1), nominate_dim1)

# Version B
congress %>%
  filter(party_code == "Republican") %>%
  select(bioname, nominate_dim1) %>%
  arrange(nominate_dim1)
```

Answer: **Version B is much easier to read because it follows a logical, left-to-right flow that matches how we think about data manipulation.** Version A requires reading from the inside out, making it hard to follow the sequence of operations. Version B reads like a recipe: “Take congress data, then filter for Republicans, then select these columns, then arrange by ideology.” The pipe operator (`%>%`) transforms nested function calls into a clear, step-by-step process that's easier to write, read, and debug.

2.3 3. Understanding DW-NOMINATE Scores

2.3.1 3.1 The Scale

DW-NOMINATE scores range from -1 to +1. What do these values represent in terms of political ideology? If a member of Congress has a score of -0.8, what does this tell you about their political positions?

Answer: **DW-NOMINATE scores measure political ideology on a liberal-conservative spectrum, where -1 represents the most liberal position and +1 represents the most conservative position.** A score of -0.8 indicates a very liberal member of Congress - they are in the left wing of the political spectrum, likely voting consistently with progressive/liberal positions on most issues. This person would be among

the most liberal members of Congress, comparable to representatives from very progressive districts.

2.3.2 3.2 Interpreting Changes

If the average DW-NOMINATE score for Democrats has moved from -0.3 in 1980 to -0.4 in 2020, what does this suggest about the party? What about if Republicans moved from +0.3 to +0.5?

Answer: **Democrats moving from -0.3 to -0.4 suggests the party has become more liberal over this 40-year period.** Republicans moving from +0.3 to +0.5 indicates they've become more conservative. **This pattern reveals increasing partisan polarization** - both parties have moved away from the center toward their respective ideological poles. This matches the common observation that American politics has become more polarized, with fewer moderate members in both parties and a widening ideological gap between them.

2.4 4. Advanced Tidyverse Operations

2.4.1 4.1 The %in% Operator

Explain what this code does and when you would use the %in% operator instead of ==:

```
congress %>%  
  filter(state_abbrev %in% c("CA", "TX", "NY", "FL"))
```

Answer: **This code filters the congress dataset to keep only rows where the state abbreviation is California, Texas, New York, or Florida.** The %in% operator checks if each value appears anywhere in the provided vector. You use %in% instead of == when you want to match against multiple values. With == you can only check one value at a time (e.g., state_abbrev == "CA"). To check multiple states with ==, you'd need: filter(state_abbrev == "CA" | state_abbrev == "TX" | state_abbrev == "NY" | state_abbrev == "FL"), which is much more verbose and error-prone.

2.4.2 4.2 Select Helpers

Match each select() helper function with what it does:

- a) starts_with("vote") 1. Selects columns containing "district" anywhere
- b) ends_with("_id") 2. Selects all remaining columns
- c) contains("district") 3. Selects columns starting with "vote"
- d) everything() 4. Selects columns ending with "_id"

Matches: a = 3, b = 4, c = 1, d = 2

2.5 5. Building Complex Analyses

2.5.1 5.1 Step-by-Step Approach

Why is it important to build your analysis step by step rather than writing complex chains all at once? Give two specific benefits.

Answer:

1. **Easier debugging:** When you build step-by-step, you can check the output at each stage to ensure it's working correctly. If something breaks in a complex chain, it's hard to identify which step caused the problem.
2. **Better understanding:** Building incrementally helps you understand what each transformation does to your data. You can examine row counts, check for unexpected NAs, and verify that filters are working as intended before adding more complexity.

2.6 6. Data Exploration Best Practices

2.6.1 6.1 Understanding Your Data

Why is it important to use functions like `count()`, `summary()`, and `glimpse()` before doing complex analysis? What might happen if you skip this step?

Answer: **These exploration functions help you understand your data's structure, quality, and characteristics before analysis, preventing serious errors.**

If you skip exploration, you might:

- **Analyze the wrong data:** Not realize you have the wrong dataset or time period
- **Mishandle data types:** Treat numeric codes as actual numbers, or text numbers as characters
- **Miss data quality issues:** Not notice impossible values, outliers, or extensive missing data
- **Make incorrect assumptions:** Assume balanced groups when they're highly imbalanced
- **Choose wrong methods:** Use techniques inappropriate for your data's distribution

Example: Without checking, you might calculate average income not realizing that -999 is used to code missing values, completely distorting your results.

2.7 7. The NOT Operator

2.7.1 7.1 Using the NOT Operator

What will each of these expressions return if `x = 5` and `y = NA`:

- a) `!TRUE = FALSE`
- b) `!(x > 3) = FALSE` (since `x > 3` is TRUE, and !TRUE is FALSE)
- c) `!is.na(x) = TRUE` (since `is.na(5)` is FALSE, and !FALSE is TRUE)
- d) `!is.na(y) = FALSE` (since `is.na(NA)` is TRUE, and !TRUE is FALSE)

2.7.2 7.2 Practical Application

Write the filter condition to keep only rows where the `vote_share` column is NOT missing:

Answer: `filter(!is.na(vote_share))`

2.8 8. Multiple Sorting

2.8.1 8.1 Understanding `arrange()` with Multiple Variables

What does this code do, and in what order will the results appear?

```
congress %>%  
  arrange(state_abbrev, desc(nominate_dim1))
```

Answer: This code sorts the congress data first alphabetically by state abbreviation (A to Z), then within each state, it sorts by DW-NOMINATE score from highest to lowest (most conservative to most liberal).

3 AI Exercises

Tips for Working with Claude:

- Ask for **R code using only tidyverse** (no other packages)
- Request **simple, focused answers** to your specific question—not complex analyses
- Ask Claude to **explain what the code is doing** since you're learning
- Avoid asking for visualizations or plots in these exercises
- Include the output of `glimpse()` in your prompt so Claude knows your variable names

Example prompt: “Using tidyverse in R, calculate the mean nominate_dim1 by party_code. Keep the code simple and explain what each line does. Here is what my data looks like: [paste glimpse output]”

3.1 9. Congressional Ideology Analysis

Dataset: data/HSall_members.csv

Description: DW-NOMINATE scores for all members of Congress throughout history.

Variables:

- congress: Congress number (int)
- chamber: House or Senate (chr)
- state_abbrev: State abbreviation (chr)
- party_code: Political party (chr)
- bioname: Member name (chr)
- nominate_dim1: Liberal-conservative score, -1 to 1 (dbl)
- nominate_dim2: Second dimension score (dbl)
- nominate_geo_mean_probability: Prediction accuracy (dbl)

3.1.1 9.1 Loading and Initial Exploration

```
# Load the dataset
library(tidyverse)
congress <- read_csv("data/HSall_members.csv")
```

Rows: 51044 Columns: 22

-- Column specification -----

Delimiter: ","

chr (5): chamber, state_abbrev, party_code, bioname, bioguide_id

dbl (16): congress, icpsr, state_icpsr, district_code, occupancy, last_means...

lgl (1): conditional

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Explore the DW-NOMINATE data
glimpse(congress)
```

```

Rows: 51,044
Columns: 22
$ congress          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ chamber           <chr> "President", "House", "House", "House", ~
$ icpsr             <dbl> 99869, 379, 4854, 6071, 1538, 2010, 3430~
$ state_icpsr       <dbl> 99, 44, 44, 44, 52, 52, 52, 52, 52, 52, ~
$ district_code     <dbl> 0, 2, 1, 3, 6, 3, 5, 2, 4, 1, 1, 3, 2, 8~
$ state_abbrev      <chr> "USA", "GA", "GA", "GA", "MD", "MD", "MD~
$ party_code        <chr> "5000", "4000", "4000", "4000", "5000", ~
$ occupancy         <dbl> NA, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ last_means        <dbl> NA, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ bioname           <chr> "WASHINGTON, George", "BALDWIN, Abraham"~
$ bioguide_id       <chr> NA, "B000084", "J000017", "M000234", "CO~
$ born              <dbl> NA, 1754, 1757, 1739, 1730, 1755, 1756, ~
$ died              <dbl> NA, 1807, 1806, 1812, 1796, 1815, 1815, ~
$ nominate_dim1     <dbl> NA, -0.165, -0.320, -0.428, 0.116, -0.08~
$ nominate_dim2     <dbl> NA, -0.373, -0.181, -0.317, -0.740, -0.3~
$ nominate_log_likelihood <dbl> NA, -28.55029, -24.89986, -12.62728, -23~
$ nominate_geo_mean_probability <dbl> NA, 0.758, 0.776, 0.880, 0.783, 0.788, 0~
$ nominate_number_of_votes <dbl> NA, 103, 98, 99, 96, 92, 94, 106, 103, 9~
$ nominate_number_of_errors <dbl> NA, 12, 9, 2, 11, 13, 10, 34, 30, 20, 10~
$ conditional       <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ nokken_poole_dim1 <dbl> NA, -0.429, -0.559, -0.413, 0.114, -0.09~
$ nokken_poole_dim2 <dbl> NA, -0.817, -0.052, -0.232, -0.779, -0.4~

```

This dataset contains ideological scores for all members of Congress. Work with Claude to understand what the `nominate_dim1` variable represents.

3.1.2 9.2 Party Polarization Over Time

Ask Claude to help you analyze how the ideological distance between Democrats and Republicans in the most recent Congress.

```

congress %>%
  filter(congress == 118) %>%
  filter(party_code %in% c("Democrat", "Republican")) %>% # Using actual party names
  group_by(party_code) %>%
  summarise(
    mean_ideology = mean(nominate_dim1, na.rm = TRUE),
    n_members = n()
  ) %>%

```

```
summarise(
  gap = max(mean_ideology) - min(mean_ideology)
)
```

```
# A tibble: 1 x 1
  gap
<dbl>
1 0.892
```

Interpretation: The analysis shows the current ideological distance between Democrats and Republicans in Congress 118. The gap represents how far apart the two parties are on the liberal-conservative spectrum, with larger values indicating greater polarization.

3.1.3 9.3 State-Level Analysis

Work with Claude to find which state has the most ideologically diverse congressional delegation in the current Congress.

```
congress %>%
  filter(congress == 118) %>%
  filter(!is.na(nominate_dim1)) %>% # Remove missing ideology scores
  filter(chamber %in% c("House", "Senate")) %>% # Only legislators, not President
  group_by(state_abbrev) %>%
  summarise(
    n_members = n(),
    mean_ideology = mean(nominate_dim1, na.rm = TRUE),
    sd_ideology = sd(nominate_dim1, na.rm = TRUE),
    min_ideology = min(nominate_dim1, na.rm = TRUE),
    max_ideology = max(nominate_dim1, na.rm = TRUE),
    range_ideology = max_ideology - min_ideology
  ) %>%
  filter(n_members > 1) %>% # Need at least 2 members to calculate diversity
  arrange(desc(sd_ideology))
```

```
# A tibble: 50 x 7
  state_abbrev n_members mean_ideology sd_ideology min_ideology max_ideology
  <chr>         <int>         <dbl>         <dbl>         <dbl>         <dbl>
1 GA              16         0.218         0.564        -0.477         0.891
2 CO              11         0.0564        0.547        -0.431         0.894
3 AZ              11         0.187         0.527        -0.598         0.818
```



```

4 TX          41      0.247      0.517      -0.783      0.826
5 WI          11      0.240      0.491      -0.524      0.643
6 VA          13      0.000385    0.477      -0.549      0.8
7 NC          16      0.153      0.461      -0.498      0.703
8 MO          10      0.424      0.445      -0.453      0.927
9 OH          18      0.205      0.441      -0.453      0.85
10 MS         6       0.339      0.430      -0.519      0.637
# i 40 more rows
# i 1 more variable: range_ideology <dbl>

```

3.2 10. School Climate and Outcomes

Dataset: data/school_climate.csv

Description: Student-level data nested in 500 schools measuring climate, discipline, academics.

Variables:

- student_id: Unique student identifier (int)
- school_id: School identifier (int)
- grade_level: 9-12 (int)
- gender: male, female, nonbinary (factor)
- race_ethnicity: Standard categories (factor)
- suspensions: Annual suspension count (int)
- gpa: 0-4 scale (num)
- school_safety: 0-10 student perception (int)
- teacher_support: 0-10 (int)
- parent_involvement: 0-10 (int)
- school_poverty_rate: % eligible for free lunch (num)

3.2.1 10.1 Data Loading and Quality Check

```

# Load the dataset
school <- read_csv("data/school_climate.csv")

```

Rows: 5000 Columns: 11

-- Column specification -----

Delimiter: ","

chr (3): school_id, gender, race_ethnicity

dbl (8): student_id, grade_level, school_poverty_rate, gpa, suspensions, sch...

i Use ``spec()`` to retrieve the full column specification for this data.
 i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
# Check the structure
glimpse(school)
```

```
Rows: 5,000
Columns: 11
$ student_id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, ~
$ school_id       <chr> "S371", "S313", "S053", "S083", "S092", "S082", "S~
$ grade_level     <dbl> 9, 12, 10, 9, 10, 12, 11, 10, 10, 11, 12, 11, 9, 1~
$ gender          <chr> "male", "female", "male", "female", "female", "fem~
$ race_ethnicity  <chr> "Other", "Black", "Other", "Black", "Latino", "Bla~
$ school_poverty_rate <dbl> 75.5, 88.0, 15.0, 89.3, 79.0, 49.2, 20.3, 80.3, 84~
$ gpa             <dbl> 2.87, 3.33, 2.72, 4.00, 1.77, 3.58, 1.84, 2.31, 2.~
$ suspensions     <dbl> 1, 0, 2, 0, 0, 0, 0, 0, 1, 0, 2, 0, 1, 0, 0, 1, 2,~
$ school_safety   <dbl> 5.6, 3.4, 5.4, 5.5, 7.3, 7.0, 9.4, 4.9, 6.1, 5.4, ~
$ teacher_support <dbl> 5.4, 6.1, 8.7, 8.2, 4.8, 5.1, 7.6, 6.5, 7.2, 6.8, ~
$ parent_involvement <dbl> 9.1, 6.2, 6.9, 3.8, 3.0, 6.7, 5.1, 6.4, 6.5, 5.1, ~
```

```
# Data quality check
school %>%
  summarise(
    total_students = n(),
    n_schools = n_distinct(school_id),
    avg_students_per_school = n() / n_distinct(school_id),
    missing_gpa = sum(is.na(gpa)),
    missing_suspensions = sum(is.na(suspensions)),
    gpa_range = paste(min(gpa, na.rm = TRUE), "-", max(gpa, na.rm = TRUE)),
    suspension_range = paste(min(suspensions, na.rm = TRUE), "-", max(suspensions, na.rm = TRUE))
  )
```

```
# A tibble: 1 x 7
  total_students n_schools avg_students_per_school missing_gpa
      <int>      <int>          <dbl>          <int>
1       5000        500            10            0
# i 3 more variables: missing_suspensions <int>, gpa_range <chr>,
#   suspension_range <chr>
```

This dataset contains student-level data nested within schools.

3.2.2 10.2 Analyzing Discipline Disparities

Ask Claude to help you look for missing data.

```
school %>%
  summarise(across(everything(), ~ sum(is.na(.x)))) %>%
  pivot_longer(everything(),
               names_to = "column",
               values_to = "missing_count")
```

```
# A tibble: 11 x 2
  column      missing_count
  <chr>      <int>
1 student_id          0
2 school_id           0
3 grade_level         0
4 gender              0
5 race_ethnicity      0
6 school_poverty_rate 0
7 gpa                 0
8 suspensions         0
9 school_safety       0
10 teacher_support    0
11 parent_involvement 0
```

Interpretation: The missing data check shows which variables have incomplete information. Understanding patterns of missingness is crucial before conducting analysis, as missing data can bias results if not handled properly.

3.2.3 10.3 School Climate and Academic Performance

Work with Claude to find mean safety perceptions and school poverty rate.

```
school %>%
  summarise(
    mean_school_safety = mean(school_safety, na.rm = TRUE),
    mean_poverty_rate = mean(school_poverty_rate, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 2
  mean_school_safety mean_poverty_rate
      <dbl>             <dbl>
1         5.99             52.4
```

Interpretation: The mean values provide a baseline understanding of typical school safety perceptions and poverty levels across the schools in the dataset. These summary statistics help contextualize individual school conditions.

3.3 11. Protest Participation Patterns

Dataset: `data/protest_panel.csv`

Description: Three-wave panel survey on protest activity & engagement.

Variables:

- `respondent_id`: Panel respondent ID (int)
- `wave`: 1, 2, 3 (int)
- `age`: Baseline age (int)
- `education`: Highest degree (factor)
- `political_interest`: 0-10, time-varying (int)
- `ideology`: 1-7 scale (int)
- `participated_protest`: Protested in past 12 months (binary)
- `protest_issue`: Racial Justice, Climate, Abortion, Economy, NA (factor)
- `social_media_use`: 0-5 daily usage index (int)
- `voted_last_election`: Turnout indicator (binary)

3.3.1 11.1 Understanding Panel Data

```
# Load the dataset
protest <- read_csv("data/protest_panel.csv")
```

```
Rows: 3000 Columns: 10
-- Column specification -----
Delimiter: ","
chr (1): protest_issue
dbl (9): respondent_id, wave, age, education, ideology, political_interest, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Examine panel structure
glimpse(protest)
```

```
Rows: 3,000
Columns: 10
$ respondent_id    <dbl> 1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6~
$ wave            <dbl> 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2~
$ age             <dbl> 78, 78, 78, 74, 74, 74, 61, 61, 61, 33, 33, 33, 7~
$ education        <dbl> 4, 4, 4, 4, 4, 4, 5, 5, 5, 2, 2, 2, 5, 5, 5, 3, 3~
$ ideology         <dbl> 2, 2, 2, 2, 2, 2, 3, 3, 3, 7, 7, 7, 4, 4, 4, 7, 7~
$ political_interest <dbl> 2, 7, 5, 3, 6, 5, 6, 5, 3, 3, 5, 4, 3, 6, 5, 6, 4~
$ social_media_use <dbl> 5, 1, 1, 3, 3, 0, 3, 3, 5, 4, 2, 5, 4, 4, 2, 2, 3~
$ participated_protest <dbl> 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0~
$ protest_issue    <chr> "Climate", NA, NA, "Racial Justice", "Economy", "~
$ voted_last_election <dbl> 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1~
```

Panel data follows the same individuals over time.

3.3.2 11.2 Who Protests?

Ask Claude to help you create a profile of protesters vs non-protesters. Which group has more males? Which group is older on average?

```
ever_protested <- protest %>%
  group_by(respondent_id) %>%
  summarise(
    ever_protested = max(participated_protest), # 1 if ever protested, 0 if never
    age = first(age) # Age is constant across waves, so take first value
  ) %>%
  mutate(protester_type = ifelse(ever_protested == 1, "Ever-Protester", "Never-Protester"))

# Compare age between the two groups
age_comparison <- ever_protested %>%
  group_by(protester_type) %>%
  summarise(
    n = n(),
    mean_age = mean(age, na.rm = TRUE),
    sd_age = sd(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    min_age = min(age, na.rm = TRUE),
```

```

    max_age = max(age, na.rm = TRUE)
  )

# Display results
cat("Age Comparison: Ever-Protesters vs Never-Protesters\n")

```

Age Comparison: Ever-Protesters vs Never-Protesters

```

cat("=====\n")

```

```

=====

```

```

print(age_comparison)

```

```

# A tibble: 2 x 7
  protester_type      n mean_age sd_age median_age min_age max_age
  <chr>          <int>   <dbl>  <dbl>     <dbl>   <dbl>   <dbl>
1 Ever-Protester    554    49.9   18.0        51      18     80
2 Never-Protester   446    48.9   18.4       48.5     18     80

```

```

# Calculate the difference
age_diff <- age_comparison$mean_age[age_comparison$protester_type == "Ever-Protester"] -
  age_comparison$mean_age[age_comparison$protester_type == "Never-Protester"]

```

Interpretation: The age comparison shows differences between those who have ever participated in protests versus those who haven't. Age can be an important factor in political participation, with different age groups having varying levels of resources, time, and motivation for protest activities.

4 Segment B — Summary Statistics

5 Non-AI Exercises

5.1 1. Understanding Measures of Central Tendency

5.1.1 1.1 Multiple Choice: Mean vs Median

When should you use the median instead of the mean?

- a) When you want the most precise calculation
- b) When the data has extreme outliers or is skewed
- c) When all values are the same
- d) When you have categorical data

Answer: **b) When the data has extreme outliers or is skewed**

Explanation: The median is robust to outliers because it represents the middle value when data is ordered. Unlike the mean, which is pulled toward extreme values, the median remains stable. For example, if incomes are \$30k, \$35k, \$40k, \$45k, and \$1 million, the mean (\$230k) is misleading while the median (\$40k) better represents the typical value.

5.1.2 1.2 Fill in the Blanks: Central Tendency

Complete these statements about measures of center:

1. The **mean** is the sum of all values divided by the count
2. The **median** is the middle value when data is ordered
3. The **mode** is the most frequently occurring value
4. When data is skewed right, the mean is typically **greater** than the median
5. The **mean** is most affected by outliers

Explanation: In right-skewed data (like income), extreme high values pull the mean upward while the median stays centered, making mean > median. The mean incorporates every value in its calculation, making it sensitive to outliers.

5.1.3 1.3 Code Detective: Grouped Summaries

What does this code calculate?

```
approval %>%
  group_by(party_id) %>%
  summarise(
    avg_approval = mean(congress_approval, na.rm = TRUE),
    n = n()
  )
```

This code calculates: **The average congressional approval rating for each political party (Democrat, Republican, Independent) and counts how many respondents are in each party group.**

Explanation: `group_by(party_id)` splits the data by party affiliation, then `summarise()` calculates the mean approval within each group. The `n()` function counts observations per group, and `na.rm = TRUE` removes missing values before calculating the mean.

5.2 2. Measures of Spread

5.2.1 2.1 Match: Measures of Spread

Match each measure with its definition:

Measures:

- a) Range
- b) Variance
- c) Standard deviation
- d) Interquartile range (IQR)

Definitions:

- 1. Square root of the variance
- 2. Maximum value minus minimum value
- 3. Average squared deviation from the mean
- 4. Distance between 25th and 75th percentiles

Matches: a = **2**, b = **3**, c = **1**, d = **4**

5.2.2 2.2 Multiple Choice: Standard Deviation

If approval ratings have a mean of 45% and standard deviation of 10%, approximately what percentage of responses fall between 35% and 55%?

- a) 50%
- b) 68%
- c) 95%
- d) 99%

Answer: b) 68%

5.2.3 2.3 True or False: Spread

Mark each statement as True (T) or False (F):

T Variance is always positive or zero **T** Standard deviation has the same units as the original data **T** A larger standard deviation means data is more spread out **T** The range is affected by outliers **T** IQR is more robust to outliers than standard deviation

Explanations: - Variance is squared deviations, so always 0 - SD is the square root of variance, returning to original units - Larger SD means values are further from the mean on average - Range uses min and max, both susceptible to outliers - IQR uses middle 50% of data, ignoring extremes

5.3 3. The summarise() Function

5.3.1 3.1 Fill in the Code: summarise()

Complete this code to calculate mean, median, and standard deviation:

```
approval %>%
  summarise(
    mean_approval = mean(congress_approval, na.rm = TRUE),
    median_approval = median(congress_approval, na.rm = TRUE),
    sd_approval = sd(congress_approval, na.rm = TRUE)
  )
```

Functions needed: **mean**, **median**, **sd**

5.3.2 3.2 Multiple Choice: NA Values

What happens when you run `mean(c(1, 2, NA, 4))`?

- a) Returns 2.33 (ignores the NA)
- b) Returns NA
- c) Gives an error
- d) Returns 2 (treats NA as 0)

Answer: **b) Returns NA**

Explanation: R propagates NA (missing) values through calculations by default. Any operation involving NA returns NA unless you explicitly tell R to remove them using `na.rm = TRUE`. This forces you to make conscious decisions about handling missing data.

5.4 4. Real-World Applications

5.4.1 4.1 Match: Statistical Concepts

Match polling concepts with statistical terms:

Polling Terms:

- a) Margin of error
- b) Sample size
- c) Confidence level
- d) Poll average

Statistical Terms:

1. Mean of multiple measurements
2. Number of observations (n)
3. Related to standard error
4. Probability of capturing true value

Matches: a = **3**, b = **2**, c = **4**, d = **1**

Explanation: Margin of error is typically $1.96 \times$ standard error for 95% confidence. Sample size (n) is the count of respondents. Confidence level (e.g., 95%) is the probability the interval contains the true population value. Poll averages aggregate multiple polls using the mean.

5.4.2 4.2 Code Detective: Real Analysis

What question does this code answer?

```
approval %>%
  group_by(party_id, region) %>%
  summarise(
    mean_approval = mean(congress_approval, na.rm = TRUE),
    sd_approval = sd(congress_approval, na.rm = TRUE),
    count = n()
  )
```

This analysis shows: **How congressional approval ratings vary by both party affiliation AND geographic region, including the average approval, spread of opinions (standard deviation), and number of respondents in each party-region combination.**

Example interpretation: We might find that Democrats in the Northeast have higher approval (mean = 55%, sd = 12%) than Republicans in the South (mean = 35%, sd = 15%), showing both partisan and regional differences.

6 AI Exercises

Tips for Working with Claude:

- Ask for **R code using only tidyverse** (no other packages)
- Request **simple, focused answers** to your specific question—not complex analyses
- Ask Claude to **explain what the code is doing** since you’re learning
- Avoid asking for visualizations or plots in these exercises
- Include the output of `glimpse()` in your prompt so Claude knows your variable names

Example prompt: “Using tidyverse in R, calculate the mean and standard deviation of `congress_approval` by `party_id`. Keep the code simple and explain what each line does. Here is what my data looks like: [paste glimpse output]”

6.1 5. Congressional Approval Analysis

Dataset: `data/congressional_approval.csv`

Description: Survey data on congressional approval ratings with demographics.

Variables:

- respondent_id: Unique identifier (int)
- age: Respondent's age (int)
- education: Education level (chr)
- party_id: Democrat, Republican, Independent (chr)
- income_category: Income bracket (chr)
- region: Geographic region (chr)
- congress_approval: Approval rating for Congress, 0-100 scale (dbl)

6.1.1 5.1 Initial Data Exploration

```
# Load the dataset
library(tidyverse)
approval <- read_csv("data/congressional_approval.csv")
```

Rows: 2000 Columns: 7

```
-- Column specification -----
Delimiter: ","
chr (4): education, party_id, income_category, region
dbl (3): respondent_id, age, congress_approval
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Examine the data
glimpse(approval)
```

Rows: 2,000

Columns: 7

```
$ respondent_id    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 1~
$ age              <dbl> 31, 68, 41, 75, 19, 55, 27, 49, 57, 56, 55, 47, 18, ~
$ education        <chr> "Some College", "Bachelor's", "Some College", "Bache~
$ party_id         <chr> "Independent", "Republican", "Republican", "Democrat~
$ income_category  <chr> "$30k-$60k", "$30k-$60k", "$30k-$60k", "$30k-$60k", ~
$ region           <chr> "Midwest", "South", "South", "South", "West", "South~
$ congress_approval <dbl> 19.132044, 19.898068, 17.585641, 52.891775, 14.57116~
```

```
summary(approval$congress_approval)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.00	19.07	28.51	28.71	38.15	75.88

Use Claude to calculate and interpret basic summary statistics for congressional approval ratings.

6.1.2 5.2 Approval by Party

Work with Claude to calculate mean and median approval ratings by party affiliation. What patterns do you observe? Are there differences in how spread out opinions are within each party?

```
# Summary statistics by party
party_summary <- approval %>%
  group_by(party_id) %>%
  summarise(
    n = n(),
    mean_approval = mean(congress_approval, na.rm = TRUE),
    median_approval = median(congress_approval, na.rm = TRUE),
    sd_approval = sd(congress_approval, na.rm = TRUE),
    cv = sd_approval / mean_approval,
    min = min(congress_approval, na.rm = TRUE),
    max = max(congress_approval, na.rm = TRUE),
    range = max - min,
    .groups = 'drop'
  ) %>%
  arrange(desc(mean_approval))

print(party_summary)
```

```
# A tibble: 3 x 9
  party_id      n mean_approval median_approval sd_approval    cv    min    max
  <chr>    <int>      <dbl>         <dbl>      <dbl> <dbl> <dbl> <dbl>
1 Democrat   682        36.5          37.4      12.2 0.333     0  73.4
2 Republican 697        27.1          26.9      12.1 0.448     0  75.9
3 Independent 621        22.0          21.5      12.2 0.553     0  64.6
# i 1 more variable: range <dbl>
```

Interpretation: Party differences in congressional approval reflect partisan polarization. The party of the current majority often shows higher approval from their base. Standard deviations reveal whether party members are unified or divided in their views.

6.1.3 5.3 Regional Variations

Ask Claude to help you explore how congressional approval varies by region. Which regions show the highest and lowest approval? Which have the most consensus (lowest standard deviation)?

```
# Regional analysis
regional_summary <- approval %>%
  group_by(region) %>%
  summarise(
    n = n(),
    mean_approval = mean(congress_approval, na.rm = TRUE),
    median_approval = median(congress_approval, na.rm = TRUE),
    sd_approval = sd(congress_approval, na.rm = TRUE),
    se_approval = sd_approval / sqrt(n),
    ci_lower = mean_approval - 1.96 * se_approval,
    ci_upper = mean_approval + 1.96 * se_approval,
    .groups = 'drop'
  ) %>%
  arrange(desc(mean_approval))

print("Regional approval rankings:")
```

```
[1] "Regional approval rankings:"
```

```
print(regional_summary)
```

```
# A tibble: 4 x 8
  region      n mean_approval median_approval sd_approval se_approval ci_lower
  <chr>   <int>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1 South     782          29.2          28.9          13.6          0.488         28.2
2 Midwest   424          28.9          28.5          12.9          0.629         27.6
3 Northeast 354          28.6          27.8          14.1          0.748         27.2
4 West      440          27.8          28.1          13.4          0.638         26.6
# i 1 more variable: ci_upper <dbl>
```

Interpretation: Regional variations in congressional approval often reflect local political cultures, economic conditions, and representation. Regions with more consensus (lower SD) may have more homogeneous political views.

6.1.4 5.4 Creating a Summary Report

Work with Claude to create a comprehensive summary table that shows approval statistics by both party and region. What insights emerge from this analysis?

```
# Comprehensive party-region analysis
party_region_summary <- approval %>%
  group_by(party_id, region) %>%
  summarise(
    n = n(),
    mean_approval = mean(congress_approval, na.rm = TRUE),
    sd_approval = sd(congress_approval, na.rm = TRUE),
    .groups = 'drop'
  ) %>%
  arrange(party_id, desc(mean_approval))

# Display as a matrix
approval_matrix <- party_region_summary %>%
  select(party_id, region, mean_approval) %>%
  pivot_wider(names_from = region, values_from = mean_approval)

print("Mean Approval by Party and Region:")
```

```
[1] "Mean Approval by Party and Region:"
```

```
print(approval_matrix)
```

```
# A tibble: 3 x 5
  party_id Northeast South Midwest West
  <chr>      <dbl> <dbl>   <dbl> <dbl>
1 Democrat    38.0  36.7   36.4  35.0
2 Independent  21.3  22.4   22.7  21.2
3 Republican  27.0  27.0   27.7  26.8
```

```
# Find extremes
extremes <- party_region_summary %>%
  mutate(combo = paste(party_id, "-", region)) %>%
  summarise(
    highest = combo[which.max(mean_approval)],
    highest_approval = max(mean_approval),
    lowest = combo[which.min(mean_approval)],
```

```

    lowest_approval = min(mean_approval),
    most_consensus = combo[which.min(sd_approval)],
    consensus_sd = min(sd_approval),
    most_divided = combo[which.max(sd_approval)],
    divided_sd = max(sd_approval)
)

print("\nKey Findings:")

```

```
[1] "\nKey Findings:"
```

```
print(paste("Highest approval:", extremes$highest, "at", round(extremes$highest_approval, 1)))
```

```
[1] "Highest approval: Democrat - Northeast at 38"
```

```
print(paste("Lowest approval:", extremes$lowest, "at", round(extremes$lowest_approval, 1)))
```

```
[1] "Lowest approval: Independent - West at 21.2"
```

```
print(paste("Most unified group:", extremes$most_consensus, "SD =", round(extremes$consensus_sd, 1)))
```

```
[1] "Most unified group: Democrat - Midwest SD = 11.3"
```

Interpretation: The intersection of party and region reveals nuanced patterns in political attitudes. Some combinations (like Democrats in liberal regions) show high approval and consensus, while others show internal divisions.

6.2 6. Income and Wealth Distribution

Dataset: data/household_panel.csv

Description: Panel data tracking household economic conditions over time.

Variables:

- hh_id: Unique household identifier (int)
- year: Year of observation (int)
- head_age: Age of household head (int)
- education_head: Education of household head (chr)

- `num_children`: Number of children (int)
- `income`: Annual household income (dbl)
- `has_employer_insurance`: Has employer insurance (lgl)
- `self_reported_health`: Health status (chr)
- `unexpected_expense`: Can handle \$400 expense (lgl)

6.2.1 6.1 Loading and Understanding the Data

```
# Load the dataset
household <- read_csv("data/household_panel.csv")
```

Rows: 6000 Columns: 9

-- Column specification -----

Delimiter: ","

dbl (9): hh_id, year, head_age, education_head, num_children, income, has_em...

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
# Check the structure
glimpse(household)
```

Rows: 6,000

Columns: 9

```
$ hh_id          <dbl> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3,~
$ year           <dbl> 2019, 2020, 2021, 2022, 2023, 2024, 2019, 2020,~
$ head_age       <dbl> 58, 58, 58, 58, 58, 58, 26, 26, 26, 26, 26, 26,~
$ education_head <dbl> 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4,~
$ num_children   <dbl> 1, 1, 0, 2, 0, 0, 0, 1, 0, 1, 3, 0, 2, 3, 2, 2,~
$ income         <dbl> 47159, 114000, 178018, 60681, 42426, 40098, 931~
$ has_employer_insurance <dbl> 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,~
$ self_reported_health <dbl> 4, 1, 4, 3, 3, 3, 3, 2, 3, 2, 3, 2, 2, 4, 4, 3,~
$ unexpected_expense <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,~
```

This is panel data (same households observed over multiple years). Ask Claude to help you understand the income and wealth distributions.

6.2.2 6.2 Income and Health

Work with Claude to explore the relationship between household income and self-reported health status. Do higher-income households report better health? Calculate appropriate summary statistics.

```
household %>%
  group_by(self_reported_health) %>%
  summarise(
    n = n(),
    mean_income = mean(income, na.rm = TRUE),
    median_income = median(income, na.rm = TRUE)
  ) %>%
  arrange(self_reported_health) %>%
  print()
```

```
# A tibble: 5 x 4
  self_reported_health      n mean_income median_income
      <dbl> <int>      <dbl>      <dbl>
1             1       291      70722.      62915
2             2       943      72501.      64183
3             3      2413      70827.      63640
4             4      1790      71950.      64678.
5             5       563      71696.      64635
```

```
household %>%
  mutate(
    income_quartile = ntile(income, 4),
    good_health = ifelse(self_reported_health >= 3, 1, 0) # Assuming 3-4 = good/excellent
  ) %>%
  group_by(income_quartile) %>%
  summarise(
    n = n(),
    pct_good_health = mean(good_health, na.rm = TRUE) * 100
  ) %>%
  print()
```

```
# A tibble: 4 x 3
  income_quartile      n pct_good_health
      <int> <int>      <dbl>
1             1    1500          79.6
```

2	2	1500	78.8
3	3	1500	79.9
4	4	1500	79.4

Interpretation: The income-health gradient is a well-documented phenomenon where higher income is associated with better health outcomes. This relationship reflects multiple factors including access to healthcare, nutrition, stress levels, and living conditions.

6.2.3 6.3 Financial Security

Ask Claude to help you analyze financial security using the `unexpected_expense` variable. What percentage of households can handle a \$400 emergency expense? How does this vary by income and education?

```
library(tidyverse)

# Overall percentage who can handle $400 emergency
household %>%
  summarise(pct_can_handle = mean(unexpected_expense == 1, na.rm = TRUE) * 100)

# A tibble: 1 x 1
  pct_can_handle
      <dbl>
1          18.5

# By income quartiles
household %>%
  mutate(income_quartile = ntile(income, 4)) %>%
  group_by(income_quartile) %>%
  summarise(pct_can_handle = mean(unexpected_expense == 1, na.rm = TRUE) * 100)

# A tibble: 4 x 2
  income_quartile pct_can_handle
      <int>          <dbl>
1           1          18.9
2           2           19
3           3          18.8
4           4          17.3
```

```
# By education level
household %>%
  group_by(education_head) %>%
  summarise(pct_can_handle = mean(unexpected_expense == 1, na.rm = TRUE) * 100) %>%
  arrange(education_head)
```

```
# A tibble: 5 x 2
  education_head pct_can_handle
      <dbl>         <dbl>
1             1             17.9
2             2             18.1
3             3             17.2
4             4             21.2
5             5             18.0
```

```
# Most vulnerable groups (lowest percentages)
household %>%
  mutate(income_quartile = ntile(income, 4)) %>%
  group_by(income_quartile) %>%
  summarise(pct_can_handle = mean(unexpected_expense == 1, na.rm = TRUE) * 100) %>%
  filter(pct_can_handle == min(pct_can_handle))
```

```
# A tibble: 1 x 2
  income_quartile pct_can_handle
      <int>         <dbl>
1           4             17.3
```

Interpretation: The ability to handle unexpected expenses is a key indicator of financial security. Large percentages of lower-income households lacking this cushion reveals financial fragility that can spiral into debt or deprivation when emergencies occur.

6.3 7. Voter Turnout Patterns

Dataset: data/voter_turnout_simple.csv

Description: State-level voter turnout data from recent elections.

Variables:

- state: State name (chr)
- turnout_2020: Turnout percentage in 2020 (dbl)
- turnout_2016: Turnout percentage in 2016 (dbl)
- population_millions: State population in millions (dbl)

6.3.1 7.1 Turnout Overview

```
# Load the dataset
turnout <- read_csv("data/voter_turnout_simple.csv")
```

```
Rows: 10 Columns: 4
-- Column specification -----
Delimiter: ","
chr (1): state
dbl (3): turnout_2020, turnout_2016, population_millions

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Explore the data
glimpse(turnout)
```

```
Rows: 10
Columns: 4
$ state      <chr> "Alabama", "Alaska", "Arizona", "Arkansas", "Calif~
$ turnout_2020 <dbl> 63.1, 58.9, 60.0, 54.9, 64.5, 76.4, 65.2, 66.0, 66~
$ turnout_2016 <dbl> 59.0, 61.5, 56.0, 53.2, 58.4, 71.9, 65.7, 61.8, 65~
$ population_millions <dbl> 5.0, 0.7, 7.3, 3.0, 39.5, 5.8, 3.6, 1.0, 21.5, 10.7
```

Work with Claude to calculate summary statistics for voter turnout in both 2016 and 2020. What changed between elections?

6.3.2 7.2 State-Level Patterns

Ask Claude to help you identify which states had the highest and lowest turnout in both elections. How consistent are state rankings between 2016 and 2020?

```
turnout %>%
  arrange(desc(turnout_2020)) %>%
  slice(1:3) %>%
  select(state, turnout_2020)
```

```
# A tibble: 3 x 2
  state      turnout_2020
  <chr>      <dbl>
1 Colorado    76.4
2 Florida     66.2
3 Georgia     66.2
```

```
turnout %>%
  arrange(turnout_2020) %>%
  slice(1:3) %>%
  select(state, turnout_2020)
```

```
# A tibble: 3 x 2
  state      turnout_2020
  <chr>      <dbl>
1 Arkansas    54.9
2 Alaska      58.9
3 Arizona      60
```

```
# Top 3 and bottom 3 states for 2016
turnout %>%
  arrange(desc(turnout_2016)) %>%
  slice(1:3) %>%
  select(state, turnout_2016)
```

```
# A tibble: 3 x 2
  state      turnout_2016
  <chr>      <dbl>
1 Colorado    71.9
2 Connecticut  65.7
3 Florida     65.3
```

```
turnout %>%
  arrange(turnout_2016) %>%
  slice(1:3) %>%
  select(state, turnout_2016)
```

```
# A tibble: 3 x 2
  state      turnout_2016
  <chr>      <dbl>
```

1	Arkansas	53.2
2	Arizona	56
3	California	58.4

Interpretation: High correlation between years suggests persistent state-level factors affecting turnout (voting laws, civic culture, demographics). States with big rank changes may have had specific mobilization efforts or law changes.

6.3.3 7.3 Turnout Changes

Work with Claude to calculate how turnout changed from 2016 to 2020 for each state. Which states saw the biggest increases or decreases?

```
turnout_changes <- turnout %>%
  mutate(
    change = turnout_2020 - turnout_2016,
    change_category = case_when(
      change >= 5 ~ "Large increase",
      change >= 2 ~ "Moderate increase",
      change >= -2 ~ "Stable",
      change >= -5 ~ "Moderate decrease",
      TRUE ~ "Large decrease"
    )
  ) %>%
  arrange(desc(change))

# Display changes by state
turnout_changes %>%
  select(state, turnout_2016, turnout_2020, change, change_category)
```

A tibble: 10 x 5

	state	turnout_2016	turnout_2020	change	change_category
	<chr>	<dbl>	<dbl>	<dbl>	<chr>
1	Georgia	59.4	66.2	6.80	Large increase
2	California	58.4	64.5	6.1	Large increase
3	Colorado	71.9	76.4	4.5	Moderate increase
4	Delaware	61.8	66	4.20	Moderate increase
5	Alabama	59	63.1	4.1	Moderate increase
6	Arizona	56	60	4	Moderate increase
7	Arkansas	53.2	54.9	1.70	Stable
8	Florida	65.3	66.2	0.900	Stable

9 Connecticut	65.7	65.2	-0.5	Stable
10 Alaska	61.5	58.9	-2.6	Moderate decrease

```
# Count states in each category
turnout_changes %>%
  count(change_category)
```

```
# A tibble: 4 x 2
  change_category      n
  <chr>           <int>
1 Large increase      2
2 Moderate decrease    1
3 Moderate increase    4
4 Stable              3
```

```
# Explore relationship with population size
correlation <- cor(turnout_changes$population_millions, turnout_changes$change)
print(paste("Correlation between population size and turnout change:", round(correlation, 3))
```

```
[1] "Correlation between population size and turnout change: 0.397"
```

```
# Show turnout changes by population size groups
turnout_changes %>%
  mutate(pop_size = case_when(
    population_millions < 2 ~ "Small (<2M)",
    population_millions < 10 ~ "Medium (2-10M)",
    TRUE ~ "Large (>10M)"
  )) %>%
  group_by(pop_size) %>%
  summarise(
    n_states = n(),
    avg_change = mean(change),
    median_change = median(change)
  )
```

```
# A tibble: 3 x 4
  pop_size      n_states avg_change median_change
  <chr>         <int>     <dbl>         <dbl>
1 Large (>10M)      3      4.60           6.1
2 Medium (2-10M)    5      2.76           4
3 Small (<2M)       2      0.800          0.800
```


Interpretation: Turnout changes reflect various factors including competitiveness, mobilization efforts, voting law changes, and demographic shifts. The relationship (or lack thereof) with population size indicates whether changes were driven by state-specific or national factors.

6.4 8. Legislative Productivity

Dataset: `data/rollcalls.csv`

Description: Individual member votes on congressional bills.

Variables:

- `congress`: Congress number (int)
- `bill_id`: Unique bill identifier (int)
- `member_id`: Member identifier (int)
- `party`: Political party (chr)
- `ideology`: Member ideology score (dbl)
- `bill_ideology`: Bill ideology score (dbl)
- `vote`: Vote choice (chr)
- `district_partisanship`: District lean (dbl)

6.4.1 8.1 Understanding Voting Patterns

```
# Load the dataset
rollcalls <- read_csv("data/rollcalls.csv")
```

```
Rows: 3000 Columns: 8
-- Column specification -----
Delimiter: ","
chr (1): party
dbl (7): congress, bill_id, member_id, ideology, bill_ideology, vote, distri...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Look at the data
glimpse(rollcalls)
```

```

Rows: 3,000
Columns: 8
$ congress      <dbl> 114, 117, 116, 114, 114, 116, 116, 118, 114, 116~
$ bill_id       <dbl> 500, 238, 494, 352, 445, 177, 621, 40, 426, 165,~
$ member_id     <dbl> 66, 409, 327, 467, 532, 487, 290, 245, 464, 90, ~
$ party         <chr> "Dem", "Rep", "Dem", "Dem", "Rep", "Rep", "Rep", ~
$ ideology      <dbl> 0.20949025, 0.41277467, -0.27790813, 0.10842921,~
$ bill_ideology  <dbl> 0.43897533, -0.33229308, -0.27514849, -0.1348527~
$ vote          <dbl> 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, ~
$ district_partisanship <dbl> 11.9, -20.8, 12.0, 15.8, -11.3, -25.6, 2.2, -13.~

```

Use Claude to calculate basic statistics about congressional voting patterns, including passage rates and participation.

6.4.2 8.2 Party Voting Patterns

Work with Claude to analyze how members vote based on their party affiliation. Do Democrats and Republicans vote differently on bills?

```

library(tidyverse)

# Calculate party voting percentages for each bill
party_voting <- rollcalls %>%
  group_by(bill_id, party) %>%
  summarise(
    total_votes = n(),
    yes_votes = sum(vote == "Yes"),
    pct_yes = mean(vote == "Yes") * 100,
    .groups = 'drop'
  ) %>%
  pivot_wider(
    names_from = party,
    values_from = c(pct_yes, total_votes),
    names_sep = "_"
  ) %>%
  mutate(
    party_split = abs(pct_yes_Dem - pct_yes_Rep)
  )

# Identify bills with strongest party splits (top 10)
strongest_splits <- party_voting %>%

```

```

arrange(desc(party_split)) %>%
slice(1:10) %>%
select(bill_id, pct_yes_Dem, pct_yes_Rep, party_split)

print("Bills with Strongest Party Splits:")

```

```
[1] "Bills with Strongest Party Splits:"
```

```
print(strongest_splits)
```

```

# A tibble: 10 x 4
  bill_id pct_yes_Dem pct_yes_Rep party_split
  <dbl>     <dbl>     <dbl>     <dbl>
1       1         0         0         0
2       2         0         0         0
3       5         0         0         0
4       6         0         0         0
5       8         0         0         0
6       9         0         0         0
7      13         0         0         0
8      14         0         0         0
9      16         0         0         0
10     17         0         0         0

```

```

# Identify bills with bipartisan agreement (both >70% or both <30%)
bipartisan <- party_voting %>%
  filter(
    (pct_yes_Dem > 70 & pct_yes_Rep > 70) | # Both parties support
    (pct_yes_Dem < 30 & pct_yes_Rep < 30)   # Both parties oppose
  ) %>%
  arrange(party_split) %>%
  slice(1:10) %>%
  select(bill_id, pct_yes_Dem, pct_yes_Rep, party_split)

print("Bills with Bipartisan Agreement:")

```

```
[1] "Bills with Bipartisan Agreement:"
```

```
print(bipartisan)
```

```
# A tibble: 10 x 4
  bill_id pct_yes_Dem pct_yes_Rep party_split
  <dbl>    <dbl>    <dbl>    <dbl>
1       1         0         0         0
2       2         0         0         0
3       5         0         0         0
4       6         0         0         0
5       8         0         0         0
6       9         0         0         0
7      13         0         0         0
8      14         0         0         0
9      16         0         0         0
10     17         0         0         0
```

```
# Overall summary of party splits
party_voting %>%
  summarise(
    mean_split = mean(party_split, na.rm = TRUE),
    median_split = median(party_split, na.rm = TRUE),
    max_split = max(party_split, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
  mean_split median_split max_split
  <dbl>    <dbl>    <dbl>
1       0         0         0
```

Interpretation: Party-line voting has increased in recent decades. Bills with high party splits often involve ideological issues, while bipartisan bills typically address non-controversial or crisis issues.

6.4.3 8.3 Ideology and Voting

Ask Claude to help you explore how member ideology relates to voting patterns. Do more extreme members vote differently than moderates?

```

# Categorize members by ideology
votes_categorized <- rollcalls %>%
  mutate(
    ideology_type = case_when(
      abs(ideology) < 0.2 ~ "Moderate",
      abs(ideology) < 0.4 ~ "Somewhat partisan",
      TRUE ~ "Extremist"
    ),
    ideology_match = ifelse(
      (ideology > 0 & bill_ideology > 0) | (ideology < 0 & bill_ideology < 0),
      "Match", "Mismatch"
    )
  )

# Compare voting patterns by ideology type
voting_by_ideology <- votes_categorized %>%
  group_by(ideology_type) %>%
  summarise(
    n_votes = n(),
    pct_yes = mean(vote == "Yes") * 100,
    avg_ideology = mean(abs(ideology))
  ) %>%
  arrange(avg_ideology)

print("Voting Patterns by Ideology Type:")

```

```
[1] "Voting Patterns by Ideology Type:"
```

```
print(voting_by_ideology)
```

```

# A tibble: 3 x 4
  ideology_type    n_votes pct_yes avg_ideology
  <chr>           <int>   <dbl>     <dbl>
1 Moderate         1131     0      0.0974
2 Somewhat partisan   932     0      0.291
3 Extremist          937     0      0.614

```

```

# Analyze voting against ideological match
votes_categorized %>%
  group_by(ideology_match) %>%

```

```

summarise(
  n_votes = n(),
  pct_yes = mean(vote == "Yes") * 100
)

# A tibble: 2 x 3
  ideology_match n_votes pct_yes
  <chr>          <int>   <dbl>
1 Match          1486     0
2 Mismatch       1514     0

# How often members vote against their ideology
against_ideology <- votes_categorized %>%
  filter(ideology_match == "Mismatch") %>%
  group_by(ideology_type) %>%
  summarise(
    pct_voted_yes_despite_mismatch = mean(vote == "Yes") * 100
  )

print("Voting Yes Despite Ideological Mismatch:")

[1] "Voting Yes Despite Ideological Mismatch:"

print(against_ideology)

# A tibble: 3 x 2
  ideology_type      pct_voted_yes_despite_mismatch
  <chr>                                <dbl>
1 Extremist          0
2 Moderate           0
3 Somewhat partisan  0

# Member-level summary: who crosses party lines most?
crossover_members <- votes_categorized %>%
  group_by(member_id, party, ideology) %>%
  summarise(
    total_votes = n(),
    mismatched_yes_votes = sum(ideology_match == "Mismatch" & vote == "Yes"),
    pct_crossover = (mismatched_yes_votes / total_votes) * 100,
    .groups = 'drop'
  )

```

```

) %>%
  arrange(desc(pct_crossover)) %>%
  slice(1:10)

print("Members Who Vote Against Ideology Most Often:")

```

```
[1] "Members Who Vote Against Ideology Most Often:"
```

```
print(crossover_members)
```

```

# A tibble: 10 x 6
  member_id party ideology total_votes mismatched_yes_votes pct_crossover
    <dbl> <chr>    <dbl>      <int>          <int>          <dbl>
1         1 Dem    -0.627         1            0            0
2         1 Dem    -0.290         1            0            0
3         1 Dem    -0.260         1            0            0
4         1 Dem     0.216         1            0            0
5         1 Dem     0.257         1            0            0
6         1 Dem     0.525         1            0            0
7         1 Rep    -0.326         1            0            0
8         2 Dem    -0.670         1            0            0
9         2 Dem    -0.564         1            0            0
10        2 Dem    -0.427         1            0            0

```

Interpretation: Ideological extremity correlates with partisan voting behavior. Moderates are more likely to cross party lines and vote based on specific bill content rather than party position. This dynamic affects legislative outcomes and coalition building.